

Conceitos de Otimização



Universidade Federal do Ceará - Campus de Quixadá

Lucas Ismaily
ismailybf@ufc.br

Semestre 2025.1

Compiladores

Baseado nos slides do Prof. Sandro Rigo (IC-Unicamp)

Introdução

- Melhorar o algoritmo é tarefa do programador.
- O compilador pode ser útil para:
 - Aplicar transformações que tornam o código gerado mais eficiente.
 - Deixar o programa livre para escrever um código limpo.

Quick Sort

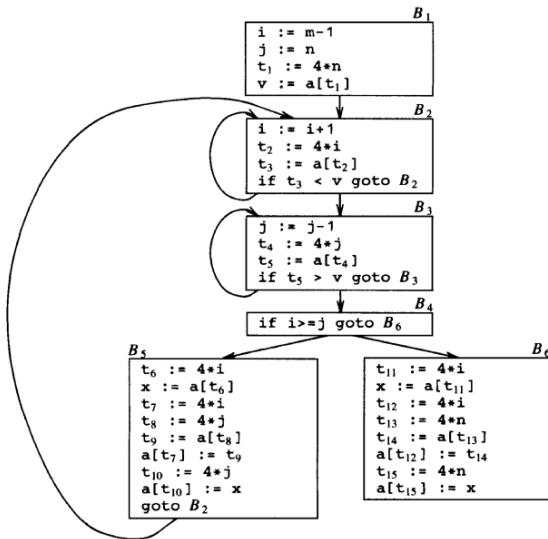
```
void quicksort(m,n)
int m,n;
{
    int i,j;
    int v,x;
    if ( n <= m ) return;
    /* o fragmento começa aqui */
    i = m-1; j = n; v = a[n];
    while(1) {
        do i = i+1; while ( a[i] < v );
        do j = j-1; while ( a[j] > v );
        if ( i >= j ) break;
        x = a[i]; a[i] = a[j]; a[j] = x;
    }
    x = a[i]; a[i] = a[n]; a[n] = x;
    /* o fragmento termina aqui */
    quicksort(m, j); quicksort(i+1, n);
}
```

Quick Sort

```
(1)  i := m-1
(2)  j := n
(3)  t1 := 4*n
(4)  v := a[t1]
(5)  i := i+1
(6)  t2 := 4*i
(7)  t3 := a[t2]
(8)  if t3 < v goto (5)
(9)  j := j-1
(10) t4 := 4*j
(11) t5 := a[t4]
(12) if t5 > v goto (9)
(13) if i >= j goto (23)
(14) t6 := 4*i
(15) x := a[t6]
```

```
(16)      t7 := 4*i
(17)      t8 := 4*j
(18)      t9 := a[t8]
(19)  a[t7] := t9
(20)      t10 := 4*j
(21)  a[t10] := x
(22)      goto (5)
(23)      t11 := 4*i
(24)      x := a[t11]
(25)      t12 := 4*i
(26)      t13 := 4*n
(27)      t14 := a[t13]
(28)  a[t12] := t14
(29)      t15 := 4*n
(30)  a[t15] := x
```

Quick Sort



Principais fontes de otimização

- Transformações que preservam a funcionalidade
 - Eliminação de Sub-expressões comuns (CSE).
 - Propagação de Cópias.
 - Eliminação de código morto.
 - Constant folding.
- Transformações locais
 - Dentro de um bloco básico.
- Transformações globais
 - Envolve mais de um bloco básico.

Principais fontes de otimização

- Muitas podem ser tanto locais como globais.
- Locais normalmente são aplicadas primeiro.

Local CSE

- E é sub-expressão comum se:
 - E foi previamente computada.
 - Os valores usados por E não sofreram alterações.

B_5

```
t6 := 4*i  
x := a[t6]  
t7 := 4*i  
t8 := 4*j  
t9 := a[t8]  
a[t7] := t9  
t10 := 4*j  
a[t10] := x  
goto B2
```

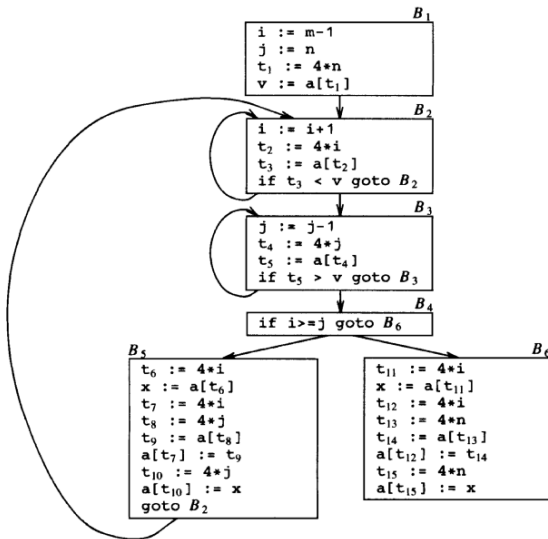
(a) Antes

B_5

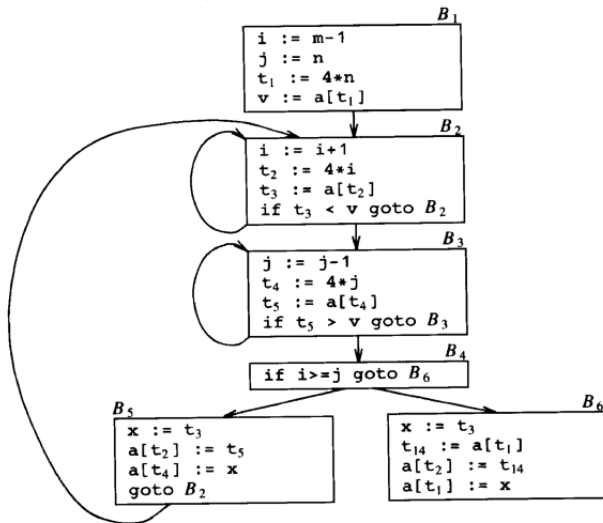
```
t6 := 4*i  
x := a[t6]  
t8 := 4*j  
t9 := a[t8]  
a[t6] := t9  
a[t8] := x  
goto B2
```

(b) Depois

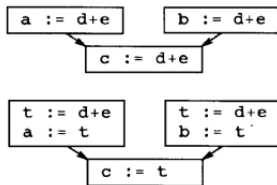
Quick Sort



Quick Sort



Propagação de cópia



- Voltemos ao bloco B5
 - Dá para melhorá-lo ainda mais?

Eliminação de código morto

- Código morto
 - Sentenças que computam valores que nunca são usados.
- Pode ser inserido
 - Pelo programador
 - `if (debug) {...}`
 - Por outras transformações:
 - Propagação de cópias.

Otimizações em Laços

- Lugar importante para otimizar
- Inner loops
 - Laços mais internos.
 - Tendem a ser onde os programas gastam a maior parte de seu tempo de execução.
- Três técnicas básicas:
 - Code Motion
 - Eliminação de variável de indução
 - Redução de força

Code Motion

- Invariantes do laço (loop-invariant)
 - Expressões que geram o mesmo resultado independente do número de iterações.
- Mover essas expressões para antes do laço
- Ex.:

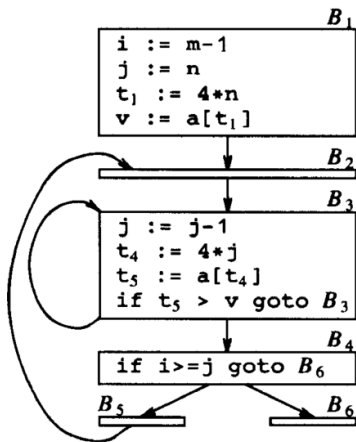
```
while (i <= limit-2)
    { ... não altera limit ... }
```

Code Motion

- Muda para:

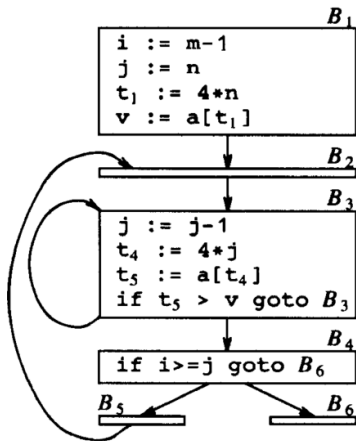
```
t = limit-2
while (t)
    { ... não altera limit,
      nem t ... }
```

Redução de força

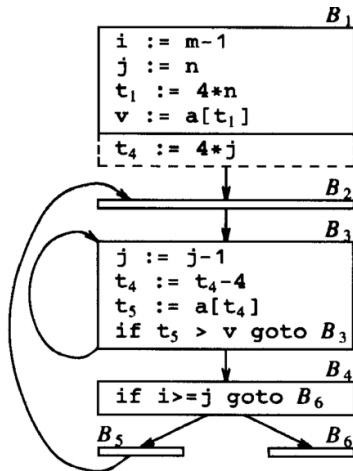


(a) Antes

Redução de força

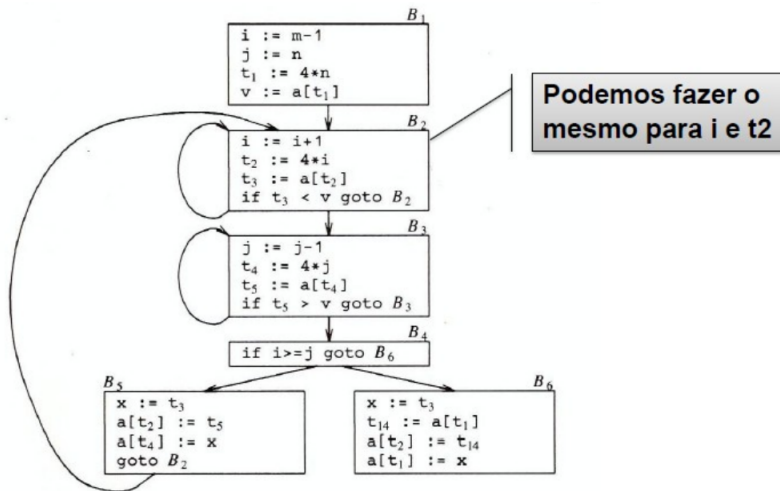


(a) Antes

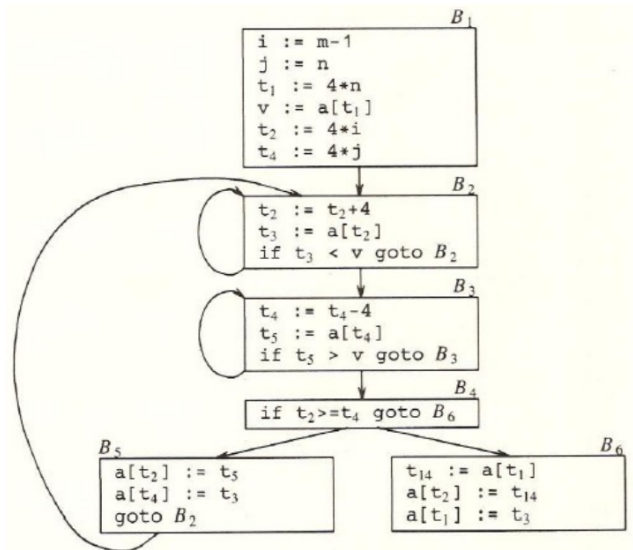


(b) Depois

Redução de força

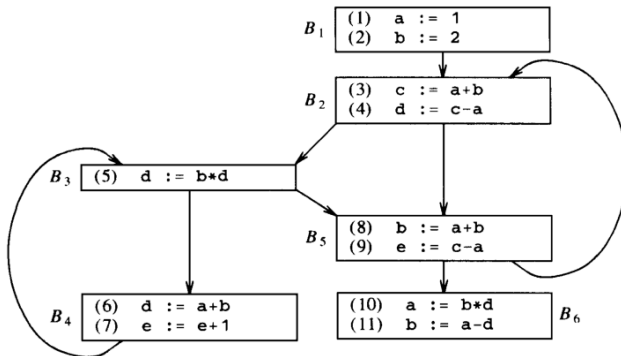


Eliminação de variável de indução



Exercício

Otimize o seguinte código:



Conceitos de Otimização



Universidade Federal do Ceará - Campus de Quixadá

Lucas Ismaily
ismailybf@ufc.br

Semestre 2025.1

Compiladores

Baseado nos slides do Prof. Sandro Rigo (IC-Unicamp)